

direct access to Google AI

Platform: Google Gemini | Generated: 5/30/2026

Sorting an array is a classic Java interview question. While it might seem straightforward, the interviewer is usually looking to see if you understand the difference between processing primitive arrays (like `int[]`) versus object arrays (like `Integer[]`), as well as the performance trade-offs between traditional loops and modern Java Streams. Here is the optimal approach for both methods, assuming we are working with a standard primitive `int[]` array.

1. Optimal Approach: Without Streams (Traditional) For pure performance, the traditional approach using `java.util.Arrays` is the most optimal. Under the hood, Java uses a highly optimized Dual-Pivot Quicksort for primitive types, which provides an excellent average time complexity of $O(N \log N)$ and operates in-place, meaning it uses $O(1)$ auxiliary space.

The CodeJava

```
import java.util.Arrays;
```

```
public class ArraySortTraditional {
    public static void main(String[] args) {
        int[] arr = {5, 2, 8, 1, 9, 3};

        // 1. Ascending Order
        Arrays.sort(arr);
        System.out.println("Ascending: " + Arrays.toString(arr));
        // Output: [1, 2, 3, 5, 8, 9]

        // 2. Descending Order
        // Note: Arrays.sort() does not take a Collections.reverseOrder() comparator for primitive int[].
        // The most optimal way to sort a primitive array in descending order without using extra space
        // is to sort it ascending first, then reverse it in-place.
        reverseArray(arr);
        System.out.println("Descending: " + Arrays.toString(arr));
        // Output: [9, 8, 5, 3, 2, 1]
    }

    // Helper method to reverse the array in-place in O(N) time and O(1) space
    private static void reverseArray(int[] arr) {
        int left = 0;
        int right = arr.length - 1;
        while (left < right) {
            int temp = arr[left];
            arr[left] = arr[right];
            arr[right] = temp;
            left++;
            right--;
        }
    }
}
```

Why this is optimal: Many developers mistakenly convert the `int[]` to an `Integer[]` just to use `Arrays.sort(arr, Collections.reverseOrder())`. While that works, boxing and unboxing every integer takes up unnecessary memory and processing time. Sorting normally and reversing with a two-pointer approach keeps your space complexity at a strict $O(1)$.

2. Optimal Approach: With Streams (Java 8+) Streams provide a declarative way to handle arrays. They are cleaner and highly readable, but it is

important to note that they carry a slight performance overhead compared to `Arrays.sort()` because they require boxing primitive values into objects to apply custom comparators. The CodeJava imports

```
java.util.Arrays;
```

```
import java.util.Collections;
```

```
public class ArraySortStreams {
    public static void main(String[] args) {
        int[] arr = {5, 2, 8, 1, 9, 3};

        // 1. Ascending Order
        int[] ascendingArr = Arrays.stream(arr)
            .sorted()
            .toArray();
        System.out.println("Ascending: " + Arrays.toString(ascendingArr));

        // 2. Descending Order
        // We must use .boxed() to convert IntStream to Stream<Integer>
        // so we can apply the Collections.reverseOrder() comparator.
        int[] descendingArr = Arrays.stream(arr)
            .boxed()
            .sorted(Collections.reverseOrder())
            .mapToInt(Integer::intValue)
            .toArray();
        System.out.println("Descending: " + Arrays.toString(descendingArr));
    }
}
```

Why this is optimal for Streams: Ascending: `Arrays.stream(arr).sorted()` directly utilizes an `IntStream`, avoiding unnecessary object creation. Descending: Because `IntStream` does not support custom comparators, you must box the primitives into `Integer` objects using `.boxed()`, sort them using `Collections.reverseOrder()`, and then unbox them back to an `int[]` using `.mapToInt(Integer::intValue)`.

This is the most idiomatic and standard way to achieve descending sort using Streams on primitives.

Summary of Complexity Approach Order Time Complexity Space

Complexity Traditional Ascending $O(N \log N)$ $O(1)$ (In-place

sorting) Traditional Descending $O(N \log N) + O(N)$ $O(1)$ (In-place reversal) Streams Ascending $O(N \log N)$ $O(N)$

(Creates a new array) Streams Descending $O(N \log N)$ $O(N)$ (Boxing overhead + new array) Interview Tip: If

asked to choose, write the Traditional approach for maximum performance and the Stream approach if the interviewer values modern, concise, and readable code. Mentioning the boxing overhead in

Streams will score you extra points.

Here is the class for sorting arrays, structured exactly like your AnagramChecker example so you can easily call `ArraySorter.run()`; from your main class. To ensure the test cases work cleanly without modifying the original array each time, I used `Arrays.copyOf()` in the traditional methods so that they return a fresh sorted array. `javapackage com.codes.java.codingquestions.arrays;`

```
import java.util.Arrays;
import java.util.Collections;

public class ArraySorter {

    // 1. Without Streams (Optimal: O(n log n) Time, O(1) Auxiliary Space for in-place sorting)
    // Note: We use Arrays.copyOf here purely so we don't mutate the original test arrays.
    public static int[] sortAscending(int[] arr) {
        if (arr == null) {
            return null;
        }
        int[] sortedArr = Arrays.copyOf(arr, arr.length);
        Arrays.sort(sortedArr);
        return sortedArr;
    }

    public static int[] sortDescending(int[] arr) {
        if (arr == null) {
            return null;
        }
        int[] sortedArr = Arrays.copyOf(arr, arr.length);
        Arrays.sort(sortedArr);

        // Reverse the array in-place
        int left = 0;
        int right = sortedArr.length - 1;
        while (left < right) {
            int temp = sortedArr[left];
            sortedArr[left] = sortedArr[right];
            sortedArr[right] = temp;
            left++;
            right--;
        }
        return sortedArr;
    }

    // 2. With Streams (Modern Approach: O(n log n) Time, O(n) Space overhead for boxing/arrays)
    public static int[] sortAscendingStream(int[] arr) {
        if (arr == null) {
            return null;
        }
    }
```

```

        return Arrays.stream(arr)
            .sorted()
            .toArray();
    }

    public static int[] sortDescendingStream(int[] arr) {
        if (arr == null) {
            return null;
        }
        return Arrays.stream(arr)
            .boxed() // Convert IntStream to Stream<Integer>
            .sorted(Collections.reverseOrder())
            .mapToInt(Integer::intValue) // Unbox back to IntStream
            .toArray();
    }

    // Test Cases
    public static void run() {
        System.out.println("\n--- Array Sorter (Ascending & Descending) ---");
        int[][] tests = {
            {5, 2, 8, 1, 9, 3},
            {10, -2, 4, 0},
            {7, 7, 7},
            {},
            null
        };

        for (int[] test : tests) {
            String original = (test == null) ? "null" : Arrays.toString(test);
            System.out.println("Input Array: " + original);

            System.out.println(" [Ascending] Traditional: " + Arrays.toString(sortAscending(test)) +
                " | Stream: " + Arrays.toString(sortAscendingStream(test)));

            System.out.println(" [Descending] Traditional: " + Arrays.toString(sortDescending(test)) +
                " | Stream: " + Arrays.toString(sortDescendingStream(test)));
            System.out.println();
        }
    }
}

```

The "Two Sum" problem is one of the most fundamental coding interview questions. It tests your ability to optimize a brute-force solution ($O(N^2)$) into a highly efficient one using appropriate data structures. The problem typically asks: Given an array of integers and a target integer, return the indices of the two numbers that add up to the target. Here is the most optimal approach using a HashMap, structured similarly to your previous classes so you can easily call it from your main class.

1. Optimal Approach: Using a HashMap (For Unsorted Arrays)

Instead of using nested loops to check every possible pair, we can iterate through the array just once. As we look at each number, we calculate its complement (target - current number). We then check if we've already seen that complement. If we have, we've found our pair! If not, we store the current number and its index in a HashMap for future reference.

The CodeJava package com.codes.java.codingquestions.arrays;

```
import java.util.Arrays;
import java.util.HashMap;
import java.util.Map;

public class TwoSum {

    // 1. HashMap Approach (Optimal for Unsorted Arrays)
    // Time: O(N), Space: O(N)
    public static int[] findTwoSum(int[] nums, int target) {
        if (nums == null || nums.length < 2) {
            return new int[]{};
        }

        // Map to store the number and its index: Map<Number, Index>
        Map<Integer, Integer> numMap = new HashMap<>();

        for (int i = 0; i < nums.length; i++) {
            int complement = target - nums[i];

            // If we have seen the complement before, we found our pair
            if (numMap.containsKey(complement)) {
                return new int[]{numMap.get(complement), i};
            }

            // Otherwise, store the current number and its index
            numMap.put(nums[i], i);
        }

        // Return empty array if no pair is found
        return new int[]{};
    }

    // 2. Two-Pointer Approach (Optimal ONLY IF the array is already sorted)
    // Note: This returns the *values* rather than original indices, because
    // sorting the array ruins the original indices.
```

```

// Time: O(N), Space: O(1)
public static int[] findTwoSumSorted(int[] nums, int target) {
    if (nums == null || nums.length < 2) {
        return new int[]{};
    }

    int left = 0;
    int right = nums.length - 1;

    while (left < right) {
        int currentSum = nums[left] + nums[right];

        if (currentSum == target) {
            return new int[]{nums[left], nums[right]};
        } else if (currentSum < target) {
            left++; // We need a larger sum, move left pointer up
        } else {
            right--; // We need a smaller sum, move right pointer down
        }
    }

    return new int[]{};
}

// Test Cases
public static void run() {
    System.out.println("\n--- Two Sum ---");

    int[] nums = {2, 7, 11, 15};
    int target = 9;

    System.out.println("Target: " + target + " | Array: " + Arrays.toString(nums));

    // Output for Unsorted/HashMap method (Returns Indices)
    int[] resultIndices = findTwoSum(nums, target);
    System.out.println("HashMap Method (Indices): " + Arrays.toString(resultIndices));
    // Expected: [0, 1]

    // Output for Sorted/Two-Pointer method (Returns Values)
    int[] resultValues = findTwoSumSorted(nums, target);
    System.out.println("Two-Pointer Method (Values): " + Arrays.toString(resultValues));
    // Expected: [2, 7]
}
}

```

Summary of Complexity Approach Array State Time Complexity Auxiliary Space Return Type

Approach	Array State	Time Complexity	Auxiliary Space	Return Type
Brute Force (Nested Loops)	Unsorted	$O(N^2)$	$O(1)$	Indices or Values
HashMap (One Pass)	Unsorted	$O(N)$	$O(N)$	Indices or Values
Two Pointers	Sorted	$O(N)$	$O(1)$	Values

Interview Tip: Always clarify with the interviewer if the array is already sorted. If they say yes, immediately jump to the Two Pointers approach, as it saves memory

($O(1)$ space). If it is not sorted, the HashMap approach is the gold standard because it trades a little bit of memory ($O(N)$) for a massive speed increase ($O(N)$ time).

"Move Zeroes" is another classic array manipulation problem (often seen as LeetCode #283). The goal is to move all 0s to the end of the array while maintaining the relative order of the non-zero elements, and crucially, doing this in-place (without creating a new array). There are two highly optimal ways to solve this. Both run in $O(N)$ time and use $O(1)$ space. I have structured the solution exactly like your previous classes.

- Optimal Approach: Single Pass (Swapping)** This is the most elegant solution. We maintain a pointer (`insertPos`) that keeps track of where the next non-zero element should go. As we iterate through the array, if we find a non-zero element, we swap it with the element at `insertPos` and move the pointer forward.
- Alternative Optimal Approach: Two Pass (Overwrite & Fill)** This approach is sometimes easier to explain in an interview. In the first pass, we simply move every non-zero element to the front of the array. In the second pass, we fill whatever is left of the array with zeroes.

The CodeJava package `com.codes.java.codingquestions.arrays`;

```
import java.util.Arrays;
```

```
public class MoveZeroes {
```

```
    // 1. Single Pass Approach (Swapping)
```

```
    // Time: O(N), Space: O(1)
```

```
    public static void moveZeroesSwap(int[] nums) {
```

```
        if (nums == null || nums.length <= 1) {
```

```
            return;
```

```
        }
```

```
        int insertPos = 0;
```

```
        for (int i = 0; i < nums.length; i++) {
```

```
            if (nums[i] != 0) {
```

```
                // Swap the non-zero element to the insert position
```

```
                int temp = nums[insertPos];
```

```
                nums[insertPos] = nums[i];
```

```
                nums[i] = temp;
```

```
                insertPos++;
```

```
            }
```

```
        }
```

```
    }
```

```
    // 2. Two Pass Approach (Overwrite then Fill)
```

```
    // Time: O(N), Space: O(1)
```

```
    public static void moveZeroesOverwrite(int[] nums) {
```

```
        if (nums == null || nums.length <= 1) {
```

```
            return;
```

```
        }
```

```
        int insertPos = 0;
```

```
        // Pass 1: Shift all non-zero elements to the front
```

```
        for (int i = 0; i < nums.length; i++) {
```

```

        if (nums[i] != 0) {
            nums[insertPos] = nums[i];
            insertPos++;
        }
    }

    // Pass 2: Fill the remaining indices with zeroes
    while (insertPos < nums.length) {
        nums[insertPos] = 0;
        insertPos++;
    }
}

// Test Cases
public static void run() {
    System.out.println("\n--- Move Zeroes ---");

    int[][] tests = {
        {0, 1, 0, 3, 12},
        {0},
        {1, 2, 3},
        {0, 0, 0, 1},
        {4, 2, 0, 1, 0}
    };

    for (int[] test : tests) {
        System.out.println("Input Array: " + Arrays.toString(test));

        // Copy arrays so the first method doesn't ruin the test case for the second
        int[] testForSwap = Arrays.copyOf(test, test.length);
        int[] testForOverwrite = Arrays.copyOf(test, test.length);

        moveZeroesSwap(testForSwap);
        moveZeroesOverwrite(testForOverwrite);

        System.out.println(" Single Pass (Swap):    " + Arrays.toString(testForSwap));
        System.out.println(" Two Pass  (Overwrite): " + Arrays.toString(testForOverwrite));
        System.out.println();
    }
}

```

Summary of Complexity	Approach	Time Complexity	Auxiliary Space	Why it works
Brute Force	$O(N)$	$O(N)$	Creates a completely new array, adds non-zeroes, then fills the rest. (Fails the "in-place" requirement).	
Two Pass (Overwrite)	$O(N)$	$O(1)$	Easy to read. Writes over the array front-to-back, then fills the tail with 0s.	
Single Pass (Swap)	$O(N)$	$O(1)$	Most optimal. Handles moving non-zeroes and placing zeroes at the end simultaneously in exactly one loop.	

Interview Tip: Start by explaining the Two Pass (Overwrite) method because the logic ("move the numbers, then fill the rest with zeroes") is extremely

intuitive. If the interviewer asks if you can do it in a single loop, smoothly transition to the Single Pass (Swap) method.

The "Missing Number" problem (LeetCode 268) asks you to find the single missing number in an array containing n distinct numbers taken from the range 0 to n . There are two highly optimal $O(N)$ time and $O(1)$ space approaches to solve this without sorting. One uses basic arithmetic, and the other uses bit manipulation. Here is the complete class formatted to fit perfectly into your project structure.

1. Optimal Approach: Math (Sum Formula) We know that the sum of the first n natural numbers can be calculated using the formula $\frac{n(n+1)}{2}$. By calculating this expected sum and subtracting the actual sum of all the numbers in the array, the remainder is our missing number.

2. Optimal Approach: Bit Manipulation (XOR) The Math approach is great, but it has a slight risk of Integer Overflow if n is exceptionally large (though standard arrays rarely hit this limit). We can avoid overflow entirely by using the XOR bitwise operator (^). XORing a number by itself results in 0 (e.g., $5 \oplus 5 = 0$). XORing by 0 leaves the number unchanged (e.g., $5 \oplus 0 = 5$).

If we XOR all the indices 0 to n and then XOR all the values in the array, every number will appear twice and cancel out to 0, leaving only the missing number.

The CodeJavapackage
com.codes.java.codingquestions.arrays;

```
import java.util.Arrays;
```

```
public class MissingNumber {
```

```
    // 1. Math Approach (Sum Formula)
```

```
    // Time: O(N), Space: O(1)
```

```
    public static int findMissingNumberMath(int[] nums) {
```

```
        if (nums == null || nums.length == 0) return -1;
```

```
        int n = nums.length;
```

```
        // Formula for the sum of the first n numbers
```

```
        int expectedSum = n * (n + 1) / 2;
```

```
        int actualSum = 0;
```

```
        for (int num : nums) {
```

```
            actualSum += num;
```

```
        }
```

```
        return expectedSum - actualSum;
```

```
    }
```

```
    // 2. Bit Manipulation Approach (XOR) - Safest against integer overflow
```

```
    // Time: O(N), Space: O(1)
```

```
    public static int findMissingNumberXOR(int[] nums) {
```

```
        if (nums == null || nums.length == 0) return -1;
```

```
        int n = nums.length;
```

```
        int missing = n; // Initialize to n because the loop below only goes up to n - 1
```

```
        for (int i = 0; i < n; i++) {
```

```
            // XOR the index and the value at that index
```

```

        missing ^= i ^ nums[i];
    }

    return missing;
}

// 3. Stream Approach (Modern Java)
// Time: O(N), Space: O(1)
public static int findMissingNumberStream(int[] nums) {
    if (nums == null || nums.length == 0) return -1;

    int n = nums.length;
    int expectedSum = n * (n + 1) / 2;

    // Sum all elements using an IntStream
    int actualSum = Arrays.stream(nums).sum();

    return expectedSum - actualSum;
}

// Test Cases
public static void run() {
    System.out.println("\n--- Missing Number ---");

    int[][] tests = {
        {3, 0, 1},           // Missing 2
        {0, 1},              // Missing 2
        {9, 6, 4, 2, 3, 5, 7, 0, 1}, // Missing 8
        {0}                  // Missing 1
    };

    for (int[] test : tests) {
        System.out.println("Input Array: " + Arrays.toString(test));

        System.out.println(" Math Method: " + findMissingNumberMath(test));
        System.out.println(" XOR Method: " + findMissingNumberXOR(test));
        System.out.println(" Stream Method: " + findMissingNumberStream(test));
        System.out.println();
    }
}

```

Summary of Complexity Approach Time Complexity Auxiliary Space Pros/Cons Sorting (Brute Force) $O(N \log N)$ $O(1)$ Too slow. HashSet (Extra Space) $O(N)$ $O(N)$ Wastes memory. Math (Sum Formula) $O(N)$ $O(1)$ Most intuitive, but risks integer overflow on massive arrays. Bitwise (XOR) $O(N)$ $O(1)$ Most optimal. Fast and completely safe from overflow. Streams $O(N)$ $O(1)$ Very concise code, but carries a slight runtime overhead compared to simple loops. Interview Tip: When presenting the Math solution, explicitly mention to the interviewer: "This is an $O(N)$ solution, but in a production environment with massive data sets, $n(n+1)/2$ could cause an integer overflow. To be perfectly safe, I

would use the XOR approach." This demonstrates deep system-level understanding beyond just knowing the algorithm.

The "Maximum Subarray" problem (often known as LeetCode 53) is a staple in coding interviews. It asks you to find the contiguous subarray (containing at least one number) which has the largest sum and return that sum. The undisputed champion for solving this is Kadane's Algorithm, which uses Dynamic Programming to solve the problem in a single pass with zero extra memory. I have provided the standard solution that returns the maximum sum, as well as a fantastic bonus implementation that returns the actual elements of the maximum subarray (interviewers love it when you can show exactly which numbers formed the sum).

1. Optimal Approach: Kadane's Algorithm (Returns Sum) The core logic of Kadane's algorithm is simple: as you iterate through the array, at each step, you decide whether to add the current element to your existing running subarray, or to start a completely new subarray from the current element. You do whichever yields a higher sum.

2. Enhanced Kadane's Algorithm (Returns the Subarray) By adding a few pointers (start, end, and tempStart), we can track exactly where the maximum subarray begins and ends without changing the time or space complexity.

The CodeJava package com.codes.java.codingquestions.arrays;

```
import java.util.Arrays;
```

```
public class MaximumSubarray {
```

```
    // 1. Standard Kadane's Algorithm (Returns the maximum sum)
```

```
    // Time: O(N), Space: O(1)
```

```
    public static int findMaxSubarraySum(int[] nums) {
```

```
        if (nums == null || nums.length == 0) {
```

```
            return 0; // Or throw an IllegalArgumentException
```

```
        }
```

```
        int currentMax = nums[0];
```

```
        int maxSoFar = nums[0];
```

```
        for (int i = 1; i < nums.length; i++) {
```

```
            // Do we extend the previous subarray, or start a new one here?
```

```
            currentMax = Math.max(nums[i], currentMax + nums[i]);
```

```
            // Did this choice give us a new global maximum?
```

```
            maxSoFar = Math.max(maxSoFar, currentMax);
```

```
        }
```

```
        return maxSoFar;
```

```
    }
```

```
    // 2. Enhanced Kadane's Algorithm (Returns the actual subarray elements)
```

```
    // Time: O(N), Space: O(1) auxiliary (O(K) to return the result array)
```

```
    public static int[] findMaxSubarrayElements(int[] nums) {
```

```
        if (nums == null || nums.length == 0) {
```

```
            return new int[]{};
```

```
        }
```

```

int currentMax = nums[0];
int maxSoFar = nums[0];

int start = 0;
int end = 0;
int tempStart = 0;

for (int i = 1; i < nums.length; i++) {
    // If the current element alone is greater than the running sum + current element,
    // we drop the running sum and start a new subarray.
    if (nums[i] > currentMax + nums[i]) {
        currentMax = nums[i];
        tempStart = i; // Mark the potential start of a new max subarray
    } else {
        currentMax += nums[i];
    }

    // If we found a new global maximum, record the bounds
    if (currentMax > maxSoFar) {
        maxSoFar = currentMax;
        start = tempStart;
        end = i;
    }
}

// Return a copy of the original array using the recorded bounds
return Arrays.copyOfRange(nums, start, end + 1);
}

// Test Cases
public static void run() {
    System.out.println("\n--- Maximum Subarray (Kadane's Algorithm) ---");

    int[][] tests = {
        {-2, 1, -3, 4, -1, 2, 1, -5, 4}, // Classic test: Max is [4, -1, 2, 1] = 6
        {1}, // Single element: Max is [1] = 1
        {5, 4, -1, 7, 8}, // All positive-ish: Max is [5, 4, -1, 7, 8] = 23
        {-5, -2, -9, -1} // All negative: Max is [-1] = -1
    };

    for (int[] test : tests) {
        System.out.println("Input Array: " + Arrays.toString(test));

        int maxSum = findMaxSubarraySum(test);
        int[] maxSubarray = findMaxSubarrayElements(test);

        System.out.println(" Max Sum: " + maxSum);
        System.out.println(" Max Subarray: " + Arrays.toString(maxSubarray));
    }
}

```



```
        System.out.println();
    }
}
}
```

Summary of Complexity Approach Time Complexity Auxiliary Space Notes
Brute Force (Nested Loops) $O(N^2)$ $O(1)$ Checks every possible subarray. Will usually result in a "Time Limit Exceeded" on coding platforms.
Divide and Conquer $O(N \log N)$ $O(\log N)$ Good to know conceptually, but slower and uses call stack memory.
Kadane's Algorithm $O(N)$ $O(1)$ Most optimal. Solves the problem dynamically by carrying a running maximum forward.
Interview Tip: A common trap interviewers set is asking, "What happens if the array contains ONLY negative numbers?" The beauty of the implementation above is that it handles all negatives perfectly. Because we initialize currentMax and maxSoFar to nums[0] (instead of 0), the algorithm will correctly identify the smallest negative number (the one closest to zero) as the maximum subarray, rather than incorrectly returning 0.

The "Best Time to Buy and Sell Stock" problem (LeetCode 121) is another classic interview staple. The goal is to maximize your profit by choosing a single day to buy one stock and choosing a different day in the future to sell it. If no profit can be achieved, you must return 0. Here is the optimal solution structured to fit into your project.

1. Optimal Approach: Single Pass (Tracking Min Price & Max Profit)

The brute-force way to solve this is to use nested loops to check the profit of every possible buy/sell combination ($O(N^2)$), but that is highly inefficient. The most optimal way is to iterate through the array just once. As we go day by day, we keep track of the lowest price we have seen so far. If the current day's price is lower than our recorded lowest price, we update our lowest price. Otherwise, we calculate how much profit we would make if we sold today, and update our maximum profit if it's the best we've seen.

The CodeJava package com.codes.java.codingquestions.arrays;

```
import java.util.Arrays;

public class BuyAndSellStock {

    // 1. Optimal Approach (Single Pass)
    // Time: O(N), Space: O(1)
    public static int findMaxProfit(int[] prices) {
        if (prices == null || prices.length < 2) {
            return 0;
        }

        int minPrice = Integer.MAX_VALUE; // Track the lowest buying price
        int maxProfit = 0;                 // Track the maximum profit we can make

        for (int i = 0; i < prices.length; i++) {
            // If we found a new cheaper buying price, update it
            if (prices[i] < minPrice) {
                minPrice = prices[i];
            }
            // Otherwise, check if selling today yields a better profit
            else if (prices[i] - minPrice > maxProfit) {
                maxProfit = prices[i] - minPrice;
            }
        }

        return maxProfit;
    }

    // Test Cases
    public static void run() {
        System.out.println("\n--- Best Time to Buy and Sell Stock ---");

        int[][] tests = {
            {7, 1, 5, 3, 6, 4}, // Buy at 1 (Day 2), Sell at 6 (Day 5) -> Profit: 5
            {7, 6, 4, 3, 1},    // Prices only drop, no profit possible -> Profit: 0
        };
    }
}
```

```

{2, 4, 1},    // Buy at 2, Sell at 4 -> Profit: 2 (Cannot sell before buying)
{2},         // Not enough days to buy and sell -> Profit: 0
{1, 2, 3, 4, 5} // Buy at 1, Sell at 5 -> Profit: 4

```

```
};
```

```
for (int[] test : tests) {
```

```
    System.out.println("Input Prices: " + Arrays.toString(test));
```

```
    System.out.println(" Max Profit: " + findMaxProfit(test));
```

```
    System.out.println();
```

```
}
```

```
}
```

```
}
```

Summary of Complexity Approach Time Complexity Auxiliary Space Why it works

Brute Force (Nested Loops) $O(N^2)$ $O(1)$ Checks every future day for every given day. Too slow for large arrays.

Single Pass (Min/Max Tracking) $O(N)$ $O(1)$ Most optimal. We only ever need to know the lowest valley we've seen in the past to calculate the peak potential profit of the current day.

Interview Tip: A common mistake candidates make is trying to find the absolute minimum and the absolute maximum of the whole array and subtracting them. However, in stocks, time only moves forward. You cannot buy at the absolute minimum if it occurs after the absolute maximum (like in the [2, 4, 1] test case). The single-pass approach naturally enforces this chronological constraint by only calculating profit against the minPrice seen prior to the current day.

The "Contains Duplicate" problem (LeetCode 217) is an excellent warm-up question in interviews. It tests your basic understanding of data structures—specifically, knowing when to trade memory space for execution speed. The problem asks you to return true if any value appears at least twice in the array, and false if every element is distinct. Here are the most common and optimal ways to solve it, structured to fit your project.

- Optimal Approach: Using a HashSet (Fastest Time)** A HashSet is designed for lightning-fast lookups ($O(1)$ average time complexity). As we iterate through the array, we try to add each number to the set. Because a Set cannot contain duplicates, the `add()` method will actually return false if the number is already in the Set. If that happens, we know we found a duplicate.
- Alternative Approach: Sorting (Best for Memory Constraints)** If the interviewer tells you that you are strictly limited on memory and cannot use the $O(N)$ extra space required by a HashSet, your best alternative is to sort the array. Once sorted, any duplicate numbers will be right next to each other, meaning you just have to check adjacent elements.

The CodeJavapackage
com.codes.java.codingquestions.arrays;

```
import java.util.Arrays;
import java.util.HashSet;
import java.util.Set;

public class ContainsDuplicate {

    // 1. HashSet Approach (Optimal for Time)
    // Time: O(N), Space: O(N)
    public static boolean containsDuplicateHashSet(int[] nums) {
        if (nums == null || nums.length <= 1) {
            return false;
        }

        Set<Integer> seen = new HashSet<>();
        for (int num : nums) {
            // Set.add() returns false if the element was already present
            if (!seen.add(num)) {
                return true;
            }
        }
        return false;
    }

    // 2. Sorting Approach (Optimal for Space, but slower)
    // Time: O(N log N), Space: O(1) auxiliary (assuming in-place sort)
    public static boolean containsDuplicateSort(int[] nums) {
        if (nums == null || nums.length <= 1) {
            return false;
        }

        // We copy the array here purely to avoid mutating the test cases.
        // In an interview, ask if you are allowed to modify the original array!
```

```

int[] sortedNums = Arrays.copyOf(nums, nums.length);
Arrays.sort(sortedNums);

for (int i = 0; i < sortedNums.length - 1; i++) {
    if (sortedNums[i] == sortedNums[i + 1]) {
        return true;
    }
}
return false;
}

// 3. Stream Approach (Modern Java, One-Liner)
// Time: O(N), Space: O(N)
public static boolean containsDuplicateStream(int[] nums) {
    if (nums == null || nums.length <= 1) {
        return false;
    }

    // If the count of distinct elements does not match the total length,
    // there must be a duplicate.
    return Arrays.stream(nums).distinct().count() != nums.length;
}

// Test Cases
public static void run() {
    System.out.println("\n--- Contains Duplicate ---");

    int[][] tests = {
        {1, 2, 3, 1},           // Contains duplicate (1) -> true
        {1, 2, 3, 4},           // All distinct -> false
        {1, 1, 1, 3, 3, 4, 3, 2, 4, 2}, // Many duplicates -> true
        {5},                     // Single element -> false
        {}                       // Empty array -> false
    };

    for (int[] test : tests) {
        System.out.println("Input Array: " + Arrays.toString(test));

        System.out.println(" HashSet Method: " + containsDuplicateHashSet(test));
        System.out.println(" Sorting Method: " + containsDuplicateSort(test));
        System.out.println(" Stream Method: " + containsDuplicateStream(test));
        System.out.println();
    }
}

```

Summary of Complexity Approach Time Complexity Auxiliary Space When to use Brute Force (Nested Loops) $O(N^2)$ $O(1)$ Never (unless asked to explain why it's bad). Sorting $O(N \log N)$ $O(1)$ Use when memory is severely constrained ($O(1)$ space requirement). HashSet $O(N)$ $O(N)$ Most optimal. The standard, expected

answer for this problem. Streams $O(N)$ $O(N)$ Use to show off modern Java knowledge, but acknowledge the runtime overhead is slightly higher than a standard loop. Interview Tip: Start with the HashSet solution. Once you finish writing it, say: "This is optimal for time, running in $O(N)$. However, if we were working in a memory-constrained environment where we couldn't afford the $O(N)$ extra space, I could optimize the memory down to $O(1)$ by sorting the array first, though it would slow our time complexity down to $O(N \log N)$." Interviewers love candidates who preemptively analyze the trade-offs between space and time.

The "Remove Duplicates" problem has two very distinct variations in interviews depending on whether the array is sorted or unsorted. The most common interview version is "Remove Duplicates from Sorted Array" (LeetCode 26), which strictly requires you to modify the array in-place with $O(1)$ extra memory and return the new length of the unique elements. If the array is unsorted and you are allowed to return a new array, a `LinkedHashSet` or `Stream` is the best approach to maintain insertion order while stripping duplicates. Here are the optimal solutions for both scenarios, structured for your project.

- Optimal Approach: Two Pointers (For Sorted Arrays, In-Place)** Because the array is already sorted, any duplicate numbers will be adjacent to each other. We can use a two-pointer approach: one pointer (`i`) scans through the array, and another pointer (`insertIndex`) keeps track of where the next unique element should be placed.
- Optimal Approach: `LinkedHashSet` (For Unsorted Arrays)** If the array is unsorted and you need to remove duplicates while preserving the original order, a `LinkedHashSet` is the gold standard. It provides $O(1)$ lookups to check for duplicates and guarantees insertion order. [The CodeJava package com.codes.java.codingquestions.arrays;](https://codejava.com/codes/java/codingquestions/arrays/)

```
import java.util.Arrays;
import java.util.LinkedHashSet;
import java.util.Set;

public class RemoveDuplicates {

    // 1. Two Pointers (Optimal for Sorted Arrays - In-Place)
    // Time: O(N), Space: O(1)
    // Returns the number of unique elements (the new "length" of the valid array)
    public static int removeDuplicatesSorted(int[] nums) {
        if (nums == null || nums.length == 0) {
            return 0;
        }

        // The first element is always unique, so we start inserting at index 1
        int insertIndex = 1;

        for (int i = 1; i < nums.length; i++) {
            // If the current element is different from the previous one, it's unique
            if (nums[i] != nums[i - 1]) {
                nums[insertIndex] = nums[i];
                insertIndex++;
            }
        }

        return insertIndex;
    }

    // 2. LinkedHashSet Approach (Optimal for Unsorted Arrays - Preserves Order)
    // Time: O(N), Space: O(N)
    // Returns a completely new array without duplicates
    public static int[] removeDuplicatesUnsorted(int[] nums) {
```

```

    if (nums == null || nums.length == 0) {
        return new int[]{};
    }

    Set<Integer> uniqueSet = new LinkedHashSet<>();
    for (int num : nums) {
        uniqueSet.add(num);
    }

    // Convert the Set back to an array
    int[] result = new int[uniqueSet.size()];
    int i = 0;
    for (int num : uniqueSet) {
        result[i++] = num;
    }
    return result;
}

// 3. Stream Approach (Modern Java for Unsorted/Sorted)
// Time: O(N), Space: O(N)
public static int[] removeDuplicatesStream(int[] nums) {
    if (nums == null || nums.length == 0) {
        return new int[]{};
    }
    return Arrays.stream(nums).distinct().toArray();
}

// Test Cases
public static void run() {
    System.out.println("\n--- Remove Duplicates ---");

    // --- Test Scenario A: Sorted Array (In-Place) ---
    int[] sortedNums = {0, 0, 1, 1, 1, 2, 2, 3, 3, 4};
    System.out.println("Input (Sorted): " + Arrays.toString(sortedNums));

    int uniqueCount = removeDuplicatesSorted(sortedNums);

    // To show the result, we only print up to the 'uniqueCount' index
    int[] validResult = Arrays.copyOfRange(sortedNums, 0, uniqueCount);
    System.out.println(" In-Place Result: " + Arrays.toString(validResult) + " | New Length: " +
uniqueCount);
    System.out.println();

    // --- Test Scenario B: Unsorted Array (Returns New Array) ---
    int[][] unsortedTests = {
        {4, 2, 4, 1, 2, 3, 1},
        {7, 7, 7, 7}
    };
};

```



```

for (int[] test : unsortedTests) {
    System.out.println("Input (Unsorted): " + Arrays.toString(test));
    System.out.println("  LinkedHashSet:  " + Arrays.toString(removeDuplicatesUnsorted(test)));
    System.out.println("  Stream Method:  " + Arrays.toString(removeDuplicatesStream(test)));
    System.out.println();
}
}

```

Summary of Complexity Approach Array State Time Complexity Auxiliary Space Behavior Two Pointers Sorted $O(N)$ $O(1)$ Most optimal. Modifies the original array and ignores the trailing "garbage" values. LinkedHashSet Unsorted $O(N)$ $O(N)$ Creates a new data structure. Perfect if you cannot mutate the original array and need to preserve the original insertion order. Streams Either $O(N)$ $O(N)$ Cleanest code. Uses `.distinct()` under the hood which manages a LinkedHashSet implicitly. Interview Tip: When solving the Two Pointer (in-place) version, interviewers often ask: "What happens to the elements at the end of the array?" Your answer should be: "They are left there as garbage values. The problem only requires us to format the first k elements of the array correctly and return k . Any values residing at indices k and beyond are ignored by the caller."

The "Rotate Array" problem (LeetCode 189) is a brilliant test of your ability to manipulate array indices in-place. The goal is to rotate an array to the right by k steps, where k is non-negative. The biggest trap in this question is when k is larger than the length of the array. For example, rotating an array of length 5 by 7 steps is exactly the same as rotating it by 2 steps. You must always calculate $k \pmod n$ first. Here are the optimal solutions.

- Optimal Approach: The 3-Step Reversal (In-Place)** This is the "magic" solution interviewers are looking for. It runs in $O(N)$ time and uses absolutely $O(1)$ extra memory. It works in three simple steps: Reverse the entire array. Reverse the first k elements. Reverse the remaining $n-k$ elements.
- Alternative Approach: Extra Array (Out-of-Place)** If the interviewer doesn't care about memory limits, you can calculate the new index for every element using the formula $(i + k) \% n$ and place the results in a brand-new array. This takes $O(N)$ space but is conceptually very straightforward.

The CodeJavapackage com.codes.java.codingquestions.arrays;

```
import java.util.Arrays;

public class RotateArray {

    // 1. Array Reversal Approach (Most Optimal: In-Place)
    // Time: O(N), Space: O(1)
    public static void rotateReversal(int[] nums, int k) {
        if (nums == null || nums.length <= 1 || k == 0) {
            return;
        }

        int n = nums.length;
        k = k % n; // Crucial: Handle cases where k > array length

        // Step 1: Reverse the entire array
        reverse(nums, 0, n - 1);

        // Step 2: Reverse the first k elements
        reverse(nums, 0, k - 1);

        // Step 3: Reverse the remaining n - k elements
        reverse(nums, k, n - 1);
    }

    // Helper method to reverse a portion of the array in-place
    private static void reverse(int[] nums, int start, int end) {
        while (start < end) {
            int temp = nums[start];
            nums[start] = nums[end];
            nums[end] = temp;
            start++;
            end--;
        }
    }
}
```

```

// 2. Extra Array Approach
// Time: O(N), Space: O(N)
public static void rotateExtraArray(int[] nums, int k) {
    if (nums == null || nums.length <= 1 || k == 0) {
        return;
    }

    int n = nums.length;
    k = k % n;
    int[] rotated = new int[n];

    for (int i = 0; i < n; i++) {
        // Calculate the new index wrapped around the array length
        rotated[(i + k) % n] = nums[i];
    }

    // Copy the rotated elements back into the original array
    for (int i = 0; i < n; i++) {
        nums[i] = rotated[i];
    }
}

// Test Cases
public static void run() {
    System.out.println("\n--- Rotate Array ---");

    int[][] tests = {
        {1, 2, 3, 4, 5, 6, 7}, // Standard case
        {-1, -100, 3, 99},    // Contains negatives
        {1, 2},               // Small array
        {1}                   // Single element
    };

    int[] ks = {3, 2, 3, 0}; // k values corresponding to the tests

    for (int i = 0; i < tests.length; i++) {
        int[] test = tests[i];
        int k = ks[i];

        System.out.println("Input Array: " + Arrays.toString(test) + " | k = " + k);

        // Copy arrays to demonstrate both methods independently
        int[] testForReversal = Arrays.copyOf(test, test.length);
        int[] testForExtraArr = Arrays.copyOf(test, test.length);

        rotateReversal(testForReversal, k);
        rotateExtraArray(testForExtraArr, k);
    }
}

```

```

        System.out.println(" Reversal Method: " + Arrays.toString(testForReversal));
        System.out.println(" Extra Array:  " + Arrays.toString(testForExtraArr));
        System.out.println();
    }
}

```

Summary of Complexity Approach Time Complexity Auxiliary Space Why it works

Brute Force (Shift 1 by 1)	Extra Array	3-Step Reversal
$O(N \times k)$	$O(N)$	$O(N)$
$O(1)$	$O(N)$	$O(1)$

You manually shift every element right by 1, and you do this entire process k times. Will cause "Time Limit Exceeded" on large inputs. Trades memory for speed. Very easy to read and understand using the modulo operator.

Most optimal. Achieves the shift entirely in-place by flipping the array around the pivot point k . Interview Tip: Start by explaining the Extra Array method out loud, specifically noting the $(i + k) \% n$ formula. Then, pivot to the Reversal Method by saying: "The extra array is great, but allocating $O(N)$ memory isn't strictly necessary. I can actually do this in-place with $O(1)$ space by reversing segments of the array..." This shows a logical progression from a good solution to a perfect one.

The "Merge Sorted Arrays" problem comes in two distinct flavors during interviews. The first variation asks you to create and return a brand new merged array. The second variation (famous as LeetCode 88) gives you one array that has a "buffer" of extra empty space at the end, and asks you to merge the second array into the first one in-place. Here are the most optimal solutions for both scenarios, structured perfectly for your project.

1. Optimal Approach: In-Place Merge (Three Pointers, Right-to-Left) If `nums1` has enough empty space at the end to accommodate `nums2`, we can achieve $O(1)$ space complexity. The trick is to start merging from the end of the arrays rather than the beginning. By placing the largest elements at the back first, we guarantee that we will never overwrite an element in `nums1` before we have had a chance to evaluate it.

2. Optimal Approach: New Array Merge (Two Pointers, Left-to-Right) If neither array has extra space, you must allocate a new array. We use two pointers starting at the beginning of each array, comparing elements one by one, and placing the smaller element into our new result array.

The CodeJava package
[com.codes.java.codingquestions.arrays](https://github.com/codesjava/codingquestions/arrays);

```
import java.util.Arrays;
import java.util.stream.IntStream;
```

```
public class MergeSortedArrays {
```

```
    // 1. In-Place Merge (Optimal for arrays with buffer space)
    // Time: O(M + N), Space: O(1)
    // 'm' is the number of valid elements in nums1. 'n' is the number of elements in nums2.
    public static void mergeInPlace(int[] nums1, int m, int[] nums2, int n) {
        int p1 = m - 1;    // Pointer for the last valid element in nums1
        int p2 = n - 1;    // Pointer for the last element in nums2
        int p = m + n - 1;  // Pointer for the very end of nums1 (the buffer zone)

        // While there are elements to compare in both arrays
        while (p1 >= 0 && p2 >= 0) {
            if (nums1[p1] > nums2[p2]) {
                nums1[p] = nums1[p1];
                p1--;
            } else {
                nums1[p] = nums2[p2];
                p2--;
            }
            p--;
        }

        // If nums2 still has elements left, copy them over.
        // (If nums1 has elements left, we do nothing because they are already in the correct place)
        while (p2 >= 0) {
            nums1[p] = nums2[p2];
            p2--;
            p--;
        }
    }
}
```

```
}
```

```
// 2. New Array Merge (Optimal for standard arrays)
```

```
// Time:  $O(M + N)$ , Space:  $O(M + N)$  to hold the result
```

```
public static int[] mergeNewArray(int[] arr1, int[] arr2) {
```

```
    if (arr1 == null) return arr2;
```

```
    if (arr2 == null) return arr1;
```

```
    int[] result = new int[arr1.length + arr2.length];
```

```
    int i = 0, j = 0, k = 0;
```

```
    // Traverse both arrays and insert the smaller element into the result
```

```
    while (i < arr1.length && j < arr2.length) {
```

```
        if (arr1[i] <= arr2[j]) {
```

```
            result[k++] = arr1[i++];
```

```
        } else {
```

```
            result[k++] = arr2[j++];
```

```
        }
```

```
    }
```

```
    // Copy any remaining elements from arr1
```

```
    while (i < arr1.length) {
```

```
        result[k++] = arr1[i++];
```

```
    }
```

```
    // Copy any remaining elements from arr2
```

```
    while (j < arr2.length) {
```

```
        result[k++] = arr2[j++];
```

```
    }
```

```
    return result;
```

```
}
```

```
// 3. Stream Approach (Modern Java)
```

```
// Time:  $O((M+N) \log(M+N))$  due to sorting, Space:  $O(M + N)$ 
```

```
public static int[] mergeStream(int[] arr1, int[] arr2) {
```

```
    if (arr1 == null) return arr2;
```

```
    if (arr2 == null) return arr1;
```

```
    // Concatenate streams and sort (note: ignores the fact that they are already sorted)
```

```
    return IntStream.concat(Arrays.stream(arr1), Arrays.stream(arr2))
```

```
        .sorted()
```

```
        .toArray();
```

```
}
```

```
// Test Cases
```

```
public static void run() {
```

```
    System.out.println("\n--- Merge Sorted Arrays ---");
```

```
// --- Scenario A: In-Place Merge ---
// nums1 has a length of 6, but only 3 valid elements.
int[] nums1 = {1, 2, 3, 0, 0, 0};
int m = 3;
int[] nums2 = {2, 5, 6};
int n = 3;

System.out.println("Scenario A: In-Place Merge (Right-to-Left)");
System.out.println(" Input nums1: " + Arrays.toString(nums1) + " | Valid: " + m);
System.out.println(" Input nums2: " + Arrays.toString(nums2) + " | Valid: " + n);

mergeInPlace(nums1, m, nums2, n);
System.out.println(" Result: " + Arrays.toString(nums1));
System.out.println();
```

```
// --- Scenario B: New Array Merge ---
int[] arr1 = {1, 3, 5, 7};
int[] arr2 = {2, 4, 6, 8, 10, 12};
```

```
System.out.println("Scenario B: Out-of-Place Merge (Left-to-Right)");
System.out.println(" Input arr1: " + Arrays.toString(arr1));
System.out.println(" Input arr2: " + Arrays.toString(arr2));
```

```
System.out.println(" Two Pointer Method: " + Arrays.toString(mergeNewArray(arr1, arr2)));
System.out.println(" Stream Method: " + Arrays.toString(mergeStream(arr1, arr2)));
```

```
}
```

```
}
Summary of ComplexityApproachArray State RequirementTime ComplexitySpace ComplexityThree
Pointers (Right-to-Left)Target array has trailing zeroes/bufferO(M+N)O(1) (Modifies target in-place)Two
Pointers (Left-to-Right)No buffer space availableO(M+N)O(M+N) (Creates a new array)StreamsNo buffer
space availableO((M+N)log(M+N))O(M+N)Interview Tip: If the interviewer gives you the In-Place
variation, immediately mention why you are starting from the end of the arrays: "If I start comparing
from the front, I'll have to shift elements continuously to avoid overwriting nums1's data, which
degrades the time complexity to O(N^2). By starting from the back where the empty zeroes are, I can
guarantee safe overwrites in a clean O(N) pass." Interviewers love it when candidates explain the why
before writing the how.
```

The "Majority Element" problem (LeetCode 169) is a brilliant algorithmic puzzle. The problem states that given an array of size n , find the majority element. The majority element is the element that appears more than $n/2$ times. You may assume that the majority element always exists in the array. While you could use a HashMap or sorting, the ultimate way to solve this is using the Boyer-Moore Voting Algorithm, which operates in $O(N)$ time and uses strictly $O(1)$ extra space. Here is the complete implementation with the most common approaches, structured for your project.

1. Optimal Approach: Boyer-Moore Voting Algorithm Think of this algorithm like a battlefield. The majority element represents one army, and all other elements represent different armies. If every soldier from a different army knocks out exactly one soldier from the majority army, the majority army will still have soldiers left over because they make up more than half the population. We maintain a candidate and a count. When the count hits 0, we pick a new candidate. If we see our candidate, the count goes up. If we see someone else, the count goes down. The candidate standing at the end is our majority element.

2. Alternative Approach: HashMap (Frequency Counting) This is the most intuitive approach. We count the occurrences of every number. If any number's count crosses the $n/2$ threshold, we return it. This takes $O(N)$ time but uses $O(N)$ space.

3. Alternative Approach: Sorting If an element appears more than half the time, then if you sort the array, that element is mathematically guaranteed to occupy the dead-center index of the array, regardless of whether it is the smallest or largest number.

The CodeJava package com.codes.java.codingquestions.arrays;

```
import java.util.Arrays;
import java.util.HashMap;
import java.util.Map;

public class MajorityElement {

    // 1. Boyer-Moore Voting Algorithm (Most Optimal)
    // Time: O(N), Space: O(1)
    public static int findMajorityBoyerMoore(int[] nums) {
        if (nums == null || nums.length == 0) {
            throw new IllegalArgumentException("Array cannot be empty");
        }

        int candidate = 0;
        int count = 0;

        for (int num : nums) {
            // If our count drops to zero, we elect a new candidate
            if (count == 0) {
                candidate = num;
            }

            // If the current number is our candidate, reinforce them.
            // Otherwise, they fight and cancel each other out.
            if (num == candidate) {
                count++;
            } else {
                count--;
            }
        }

        return candidate;
    }
}
```



```

        count--;
    }
}

// Note: The problem guarantees a majority element exists.
// If it didn't, we would need a second pass here to verify
// that our 'candidate' actually appears more than n/2 times.
return candidate;
}

```

```

// 2. HashMap Approach (Good for interviews if you forget Boyer-Moore)
// Time: O(N), Space: O(N)

```

```

public static int findMajorityHashMap(int[] nums) {
    if (nums == null || nums.length == 0) {
        throw new IllegalArgumentException("Array cannot be empty");
    }

    int majorityThreshold = nums.length / 2;
    Map<Integer, Integer> counts = new HashMap<>();

    for (int num : nums) {
        counts.put(num, counts.getDefault(num, 0) + 1);

        // As soon as we cross the threshold, return it
        if (counts.get(num) > majorityThreshold) {
            return num;
        }
    }

    return -1;
}

```

```

// 3. Sorting Approach (Simplest code)
// Time: O(N log N), Space: O(1) or O(N) depending on the sorting algorithm
public static int findMajoritySorting(int[] nums) {
    if (nums == null || nums.length == 0) {
        throw new IllegalArgumentException("Array cannot be empty");
    }

    // Sort the array (modifies original array, ask interviewer if this is allowed!)
    Arrays.sort(nums);

    // The majority element will always be at the middle index
    return nums[nums.length / 2];
}

```

```

// Test Cases
public static void run() {

```

```
System.out.println("\n--- Majority Element ---");
```

```
int[][] tests = {  
    {3, 2, 3},          // Majority: 3  
    {2, 2, 1, 1, 1, 2, 2}, // Majority: 2  
    {6, 5, 5},          // Majority: 5  
    {1}                 // Majority: 1  
};
```

```
for (int[] test : tests) {  
    System.out.println("Input Array: " + Arrays.toString(test));  
  
    // Note: We don't copy arrays here except for the sorting method to prevent mutation  
    int[] testForSort = Arrays.copyOf(test, test.length);  
  
    System.out.println(" Boyer-Moore: " + findMajorityBoyerMoore(test));  
    System.out.println(" HashMap:   " + findMajorityHashMap(test));  
    System.out.println(" Sorting:   " + findMajoritySorting(testForSort));  
    System.out.println();  
}  
}
```

Summary of Complexity Approach Time Complexity Auxiliary Space Why it works / Trade-offs

Sorting	$O(N \log N)$	$O(1)$	Super easy to write (literally a 2-liner), but relatively slow. Might alter the original array.
HashMap	$O(N)$	$O(N)$	Fast and logically straightforward, but fails if the interviewer restricts memory usage.
Boyer-Moore	$O(N)$	$O(1)$	Most optimal. Achieves a linear scan with absolutely zero extra memory allocations.

Interview Tip: When you write the Boyer-Moore solution, the interviewer will often ask a follow-up: "What if the array is NOT guaranteed to have a majority element?" Your answer should be: "Boyer-Moore will always return a candidate, but if there's no guarantee, that candidate might be a false positive. To fix this, I would just add a second $O(N)$ loop at the very end to count the occurrences of the surviving candidate and verify that its count is strictly greater than $n / 2$."

Finding the largest and second-largest elements in an array is a common warm-up question. While the premise sounds incredibly simple, interviewers use this question to see if you remember to handle edge cases—specifically, what happens if the array has duplicate highest numbers (like [5, 5, 4]) or if it contains only negative numbers. Here are the most optimal ways to solve it, structured to fit perfectly into your ongoing project.

1. Optimal Approach: Single Pass (Tracking Both) The most efficient way to solve this is to iterate through the array exactly once. We maintain two variables, `largest` and `secondLargest`. If we find a number bigger than `largest`, the old `largest` gets demoted to `secondLargest`, and the new number takes the top spot. If we find a number that is smaller than `largest` but bigger than `secondLargest`, we just update `secondLargest`.

2. Stream Approach: Distinct and Sort Using Java Streams, we can filter out duplicates (`distinct()`), sort the stream in descending order, skip the first element, and grab the second one. While highly readable, this carries the overhead of sorting.

The CodeJava package `com.codes.java.codingquestions.arrays`;

```
import java.util.Arrays;
import java.util.Comparator;

public class LargestAndSecondLargest {

    // 1. Optimal Approach (Single Pass)
    // Time: O(N), Space: O(1)
    public static int[] findLargestAndSecondLargest(int[] nums) {
        if (nums == null || nums.length < 2) {
            throw new IllegalArgumentException("Array must contain at least two elements.");
        }

        int largest = Integer.MIN_VALUE;
        int secondLargest = Integer.MIN_VALUE;

        for (int num : nums) {
            if (num > largest) {
                // The current largest gets demoted to second largest
                secondLargest = largest;
                largest = num;
            } else if (num > secondLargest && num != largest) {
                // We found a number strictly smaller than the largest,
                // but bigger than our current second largest
                secondLargest = num;
            }
        }

        // If secondLargest is still MIN_VALUE, it means all elements were identical
        if (secondLargest == Integer.MIN_VALUE) {
            System.out.println(" Warning: No distinct second largest element exists.");
        }

        return new int[]{largest, secondLargest};
    }
}
```

```

}

// 2. Stream Approach (Modern Java)
// Time: O(N log N) due to sorting, Space: O(N) for boxing/streams
public static int[] findLargestAndSecondLargestStream(int[] nums) {
    if (nums == null || nums.length < 2) {
        throw new IllegalArgumentException("Array must contain at least two elements.");
    }

    int largest = Arrays.stream(nums).max().orElse(Integer.MIN_VALUE);

    // Find second largest by making elements distinct, sorting descending, and skipping 1
    int secondLargest = Arrays.stream(nums)
        .boxed()
        .distinct()
        .sorted(Comparator.reverseOrder())
        .skip(1)
        .findFirst()
        .orElse(Integer.MIN_VALUE);

    return new int[]{largest, secondLargest};
}

// Test Cases
public static void run() {
    System.out.println("\n--- Largest and Second Largest Element ---");

    int[][] tests = {
        {12, 35, 1, 10, 34, 1}, // Standard case: Largest 35, Second 34
        {10, 5, 10},           // Duplicate highest: Largest 10, Second 5
        {7, 7, 7},             // All identical elements (Edge case)
        {-10, -40, -20, -5}    // All negative elements
    };

    for (int[] test : tests) {
        System.out.println("Input Array: " + Arrays.toString(test));

        int[] resultTraditional = findLargestAndSecondLargest(test);
        int[] resultStream = findLargestAndSecondLargestStream(test);

        System.out.println(" Traditional Method: Largest = " + resultTraditional[0] +
            ", Second = " + (resultTraditional[1] == Integer.MIN_VALUE ? "None" :
resultTraditional[1]));
        System.out.println(" Stream Method:   Largest = " + resultStream[0] +
            ", Second = " + (resultStream[1] == Integer.MIN_VALUE ? "None" : resultStream[1]));
        System.out.println();
    }
}

```

```

}
Summary of ComplexityApproachTime ComplexityAuxiliary SpaceWhy it works / Trade-offsSorting
(Brute Force) $O(N \log N)$  $O(1)$ Sort array, loop backward to find the first number smaller than the last
element. Too slow.Single Pass $O(N)$  $O(1)$ Most optimal. Uses only two integers in memory and evaluates
everything in exactly one loop.Streams $O(N \log N)$  $O(N)$ Easy to read, but the .sorted() method forces an
 $O(N \log N)$  runtime.Interview Tip: Pay close attention to the line num != largest in the else if condition of
the optimal approach. If you forget to include this, an array like [10, 10, 5] will incorrectly update the
second largest to 10. Interviewers are specifically looking to see if you catch that a "second largest"
element must be strictly less than the largest element.

```

This is a fantastic twist on the standard "Two Sum" or "Largest Pair" interview questions. It tests your ability to think mathematically about combinations and edge cases (like duplicate numbers). To find the second highest distinct sum of a pair, you do not need to generate every possible pair sum ($O(N^2)$). You only ever need to find the three largest elements in the array. Here is why:

If we have the top three largest numbers in descending order ($A \geq B \geq C$), the largest possible sum is always $A+B$. The second largest distinct sum will ALWAYS be either $A+C$ or $B+C$. No other combination in the entire array can yield a higher sum than those. Here are the most optimal ways to solve this, formatted for your project.

- Optimal Approach: Single Pass (Top 3 Tracking)** We can find the three largest elements in exactly one pass ($O(N)$ time). To avoid bugs with `Integer.MIN_VALUE` or integer underflow with negative numbers, we can simply initialize our top three trackers using the first three elements of the array, then iterate through the rest.
- Alternative Approach: Sorting** If memory is strictly limited but you want the easiest, most readable code, you can sort the array in $O(N \log N)$ time. Once sorted, the top three elements are simply sitting at the very end of the array.

The CodeJava package `com.codes.java.codingquestions.arrays`;

```
import java.util.Arrays;
```

```
public class SecondHighestSumPair {
```

```
    // 1. Single Pass Approach (Optimal)
```

```
    // Time: O(N), Space: O(1)
```

```
    public static int[] findSecondHighestSumPair(int[] nums) {
```

```
        // We need at least 3 elements to guarantee a strictly *second* highest distinct sum.
```

```
        // (An array of 2 elements only has 1 sum).
```

```
        if (nums == null || nums.length < 3) {
```

```
            return new int[]{};
```

```
        }
```

```
        // Initialize the top 3 using the first 3 elements to avoid Integer.MIN_VALUE bugs
```

```
        int first = Math.max(nums[0], Math.max(nums[1], nums[2]));
```

```
        int third = Math.min(nums[0], Math.min(nums[1], nums[2]));
```

```
        int second = nums[0] + nums[1] + nums[2] - first - third; // Mathematical trick to get the middle
```

```
        // Scan the rest of the array to maintain the top 3 largest elements
```

```
        for (int i = 3; i < nums.length; i++) {
```

```
            int num = nums[i];
```

```
            if (num > first) {
```

```
                third = second;
```

```
                second = first;
```

```
                first = num;
```

```
            } else if (num > second) {
```

```
                third = second;
```

```
                second = num;
```

```
            } else if (num > third) {
```

```
                third = num;
```

```
            }
```

```

}

// Calculate the maximum possible sum
int maxSum = first + second;
int secondMaxSum = Integer.MIN_VALUE;
int[] resultPair = new int[]{};

// The second highest sum must be either (first + third) or (second + third)
if (first + third < maxSum) {
    secondMaxSum = first + third;
    resultPair = new int[]{first, third};
}

if (second + third < maxSum && second + third > secondMaxSum) {
    // secondMaxSum = second + third; // Only needed if you return the sum
    resultPair = new int[]{second, third};
}

return resultPair;
}

```

// 2. Sorting Approach

// Time: O(N log N), Space: O(1)

```

public static int[] findSecondHighestSumPairSorted(int[] nums) {
    if (nums == null || nums.length < 3) {
        return new int[]{};
    }

```

// Copy array so we don't mutate the original test cases

```

int[] sortedNums = Arrays.copyOf(nums, nums.length);
Arrays.sort(sortedNums);

```

```

int n = sortedNums.length;
int first = sortedNums[n - 1];
int second = sortedNums[n - 2];
int third = sortedNums[n - 3];

```

```

int maxSum = first + second;
int secondMaxSum = Integer.MIN_VALUE;
int[] resultPair = new int[]{};

```

```

if (first + third < maxSum) {
    secondMaxSum = first + third;
    resultPair = new int[]{first, third};
}

```

```

if (second + third < maxSum && second + third > secondMaxSum) {
    resultPair = new int[]{second, third};
}

```

```

    return resultPair;
}

// Test Cases
public static void run() {
    System.out.println("\n--- Find Pair with Second Highest Sum ---");

    int[][] tests = {
        {10, 8, 5, 2},    // Top 3: 10, 8, 5. Max: 18. Second Max: 15. Pair: [10, 5]
        {10, 9, 9, 8},    // Top 3: 10, 9, 9. Max: 19. Second Max: 18. Pair: [9, 9]
        {10, 10, 9, 8},   // Top 3: 10, 10, 9. Max: 20. Second Max: 19. Pair: [10, 9]
        {7, 7, 7},        // All same. Max: 14. Second Max: None. Pair: []
        {-5, -2, -10, -1}, // Negatives. Top 3: -1, -2, -5. Max: -3. Second Max: -6. Pair: [-1, -5]
        {4, 5}            // Less than 3 elements. Pair: []
    };

    for (int[] test : tests) {
        System.out.println("Input Array: " + Arrays.toString(test));

        int[] resultOptimal = findSecondHighestSumPair(test);
        int[] resultSorted = findSecondHighestSumPairSorted(test);

        System.out.println(" Single Pass (O(N)): " + Arrays.toString(resultOptimal));
        System.out.println(" Sorting (O(N log N)): " + Arrays.toString(resultSorted));
        System.out.println();
    }
}

```

Summary of Complexity Approach Time Complexity Auxiliary Space Notes Brute

Force $O(N^2)$ $O(1)$ Generates every possible pair using nested loops, sorts the sums, and finds the second highest. Very inefficient. Sorting $O(N \log N)$ $O(1)$ Easy to implement and mathematically identical to the $O(N)$ approach, but loses points for execution time. Top 3 Tracking (Single Pass) $O(N)$ $O(1)$ Most optimal. Pulls the three necessary elements in exactly one pass with zero extra allocations. Interview Tip: Start by explaining the math: "To find the highest sum, I need the top two elements. To find the distinct second-highest sum, I only need to evaluate pairs made from the top three elements." Once the interviewer agrees with that logic, the code becomes incredibly easy to defend. Make sure to point out how you initialize first, second, and third using indices 0, 1, and 2 to explicitly avoid Integer.MIN_VALUE bugs when dealing with negative arrays.

The "Intersection of Two Arrays" problem (LeetCode 349) asks you to find the common elements between two arrays. The key requirement here is that each element in the result must be unique, and you can return the result in any order. There are two optimal ways to solve this, depending on whether you want to optimize for time (speed) or space (memory).

1. Optimal Approach for Time: Two HashSets
This is the fastest approach. We convert the first array into a HashSet to remove its duplicates and achieve $O(1)$ lookups. Then, we iterate through the second array, checking if each element exists in our first set. If it does, we add it to an intersectionSet to guarantee our final answer only contains unique numbers.

2. Optimal Approach for Space: Sorting & Two Pointers
If the interviewer restricts your memory to $O(1)$ extra space, you cannot use HashSets. Instead, you can sort both arrays and use two pointers to find matching elements. (Note: Sorting usually takes $O(\log N)$ space depending on the language's internal sort algorithm, but in interview contexts, it is often accepted as the in-place alternative).

The CodeJava package `com.codes.java.codingquestions.arrays`;

```
import java.util.Arrays;
import java.util.HashSet;
import java.util.Set;

public class ArrayIntersection {

    // 1. HashSet Approach (Optimal for Time)
    // Time: O(M + N), Space: O(M) where M is the length of nums1
    public static int[] intersectionHashSet(int[] nums1, int[] nums2) {
        if (nums1 == null || nums2 == null || nums1.length == 0 || nums2.length == 0) {
            return new int[]{};
        }

        Set<Integer> set1 = new HashSet<>();
        for (int num : nums1) {
            set1.add(num);
        }

        Set<Integer> intersectionSet = new HashSet<>();
        for (int num : nums2) {
            if (set1.contains(num)) {
                intersectionSet.add(num);
            }
        }

        // Convert the Set back to an int[] array
        int[] result = new int[intersectionSet.size()];
        int i = 0;
        for (int num : intersectionSet) {
            result[i++] = num;
        }

        return result;
    }
}
```

```
}
```

```
// 2. Sorting & Two Pointers Approach (Optimal for Memory constraints)
```

```
// Time:  $O(M \log M + N \log N)$ , Space:  $O(1)$  auxiliary (ignoring result array)
```

```
public static int[] intersectionTwoPointers(int[] nums1, int[] nums2) {  
    if (nums1 == null || nums2 == null || nums1.length == 0 || nums2.length == 0) {  
        return new int[]{};  
    }  
  
    // Copying arrays purely so we don't mutate the test cases  
    int[] sorted1 = Arrays.copyOf(nums1, nums1.length);  
    int[] sorted2 = Arrays.copyOf(nums2, nums2.length);  
  
    Arrays.sort(sorted1);  
    Arrays.sort(sorted2);  
  
    Set<Integer> intersectionSet = new HashSet<>();  
    int i = 0;  
    int j = 0;  
  
    while (i < sorted1.length && j < sorted2.length) {  
        if (sorted1[i] < sorted2[j]) {  
            i++;  
        } else if (sorted1[i] > sorted2[j]) {  
            j++;  
        } else {  
            // They match! Add to our unique set and move both pointers  
            intersectionSet.add(sorted1[i]);  
            i++;  
            j++;  
        }  
    }  
}
```

```
}
```

```
// 3. Stream Approach (Modern Java)
```

```
// Time:  $O(M + N)$ , Space:  $O(M)$ 
```

```
public static int[] intersectionStream(int[] nums1, int[] nums2) {  
    if (nums1 == null || nums2 == null) return new int[]{};  
}
```

```

// Convert nums1 to a List/Set for the stream filter to use
Set<Integer> set1 = new HashSet<>();
for (int num : nums1) set1.add(num);

return Arrays.stream(nums2)
    .distinct()
    .filter(set1::contains)
    .toArray();
}

// Test Cases
public static void run() {
    System.out.println("\n--- Find Intersection of Two Arrays ---");

    int[][] tests = {
        {{1, 2, 2, 1}, {2, 2}}, // Standard duplicate case -> [2]
        {{4, 9, 5}, {9, 4, 9, 8, 4}}, // Standard mixed case -> [4, 9] or [9, 4]
        {{1, 2, 3}, {4, 5, 6}}, // No intersection -> []
        {{1, 1, 1}, {1, 1}} // All same elements -> [1]
    };

    for (int[][] test : tests) {
        int[] nums1 = test[0];
        int[] nums2 = test[1];

        System.out.println("Input nums1: " + Arrays.toString(nums1));
        System.out.println("Input nums2: " + Arrays.toString(nums2));

        System.out.println(" HashSet Method: " + Arrays.toString(intersectionHashSet(nums1,
nums2)));
        System.out.println(" Two Pointers Method: " + Arrays.toString(intersectionTwoPointers(nums1,
nums2)));
        System.out.println(" Stream Method: " + Arrays.toString(intersectionStream(nums1,
nums2)));
        System.out.println();
    }
}

```

Summary of Complexity Approach Time Complexity Auxiliary Space When to use Two HashSets $O(M+N)$ $O(\min(M,N))$ Most optimal overall. Trades a tiny bit of memory for the fastest possible execution time. Sorting + Two Pointers $O(M \log M + N \log N)$ $O(1)$ Use only when the arrays are already sorted for you (making time $O(M+N)$), or if memory is strictly limited. Streams $O(M+N)$ $O(M)$ Shortest code, but boxing/unboxing overhead makes it slightly slower than the manual HashSet approach. Interview Tip: A very common follow-up question is "Intersection of Two Arrays II" (LeetCode 350). The interviewer will ask: "What if the result needs to include duplicates? For example, if $\text{nums1} = [2, 2]$ and $\text{nums2} = [1, 2, 2, 1]$, the answer should be $[2, 2]$ instead of just $[2]$." If they ask this, you just need to explain that you would switch from a HashSet to a HashMap. You would count the frequencies

of the numbers in `nums1`, and as you iterate through `nums2`, if the number is in the map and the count is >0 , you add it to the result and decrement the count in the map!

The "Product of Array Except Self" problem (LeetCode 238) is a brilliant test of algorithmic thinking. The catch that makes this problem interesting is the standard interview constraint: You must write an algorithm that runs in $O(N)$ time and without using the division operation. If division were allowed, you could just multiply all the numbers together and divide by the current element (handling zeroes as edge cases). Since division is banned, we must use Prefix and Suffix Products. Here is the optimal approach, formatted to perfectly match your ongoing project structure.

1. Optimal Approach: Left/Right Running Products (In-Place)

Instead of allocating two separate arrays to hold the "products to the left" and "products to the right", we can do this in $O(1)$ extra space by using our final output array to hold the left products.

Pass 1 (Left to Right): Fill the output array with the running product of all elements to the left of the current index.

Pass 2 (Right to Left): Traverse backward. Keep a running tally of the product of all elements to the right. Multiply this running tally directly into the output array.

The Code

Java package com.codes.java.codingquestions.arrays;

```
import java.util.Arrays;
```

```
public class ProductExceptSelf {
```

```
    // 1. Optimal Approach (O(N) Time, O(1) Extra Space)
```

```
    // Note: The output array does not count as extra space for complexity analysis.
```

```
    public static int[] productExceptSelfOptimal(int[] nums) {
```

```
        if (nums == null || nums.length == 0) {
```

```
            return new int[]{};
```

```
        }
```

```
        int n = nums.length;
```

```
        int[] result = new int[n];
```

```
        // Step 1: Calculate left products
```

```
        // Initialize the first element as 1 since there is nothing to the left of index 0
```

```
        int leftProduct = 1;
```

```
        for (int i = 0; i < n; i++) {
```

```
            result[i] = leftProduct;
```

```
            leftProduct *= nums[i]; // Update running left product for the NEXT index
```

```
        }
```

```
        // Step 2: Calculate right products and multiply on the fly
```

```
        // Initialize the right product as 1 since there is nothing to the right of the last index
```

```
        int rightProduct = 1;
```

```
        for (int i = n - 1; i >= 0; i--) {
```

```
            // Multiply the stored left product by the running right product
```

```
            result[i] *= rightProduct;
```

```
            rightProduct *= nums[i]; // Update running right product for the NEXT index
```

```
        }
```

```
        return result;
```

```
    }
```

```

// 2. Alternative Approach: Two Arrays (O(N) Time, O(N) Space)
// This is mathematically the same, but easier to explain if you get stuck.
public static int[] productExceptSelfTwoArrays(int[] nums) {
    if (nums == null || nums.length == 0) return new int[]{};

    int n = nums.length;
    int[] leftProducts = new int[n];
    int[] rightProducts = new int[n];
    int[] result = new int[n];

    leftProducts[0] = 1;
    for (int i = 1; i < n; i++) {
        leftProducts[i] = leftProducts[i - 1] * nums[i - 1];
    }

    rightProducts[n - 1] = 1;
    for (int i = n - 2; i >= 0; i--) {
        rightProducts[i] = rightProducts[i + 1] * nums[i + 1];
    }

    for (int i = 0; i < n; i++) {
        result[i] = leftProducts[i] * rightProducts[i];
    }

    return result;
}

// Test Cases
public static void run() {
    System.out.println("\n--- Product of Array Except Self ---");

    int[][] tests = {
        {1, 2, 3, 4},    // Standard case -> [24, 12, 8, 6]
        {-1, 1, 0, -3, 3}, // Contains a single zero -> [0, 0, 9, 0, 0]
        {0, 4, 0},        // Contains multiple zeroes -> [0, 0, 0]
        {5, 9}            // Two elements -> [9, 5]
    };

    for (int[] test : tests) {
        System.out.println("Input Array: " + Arrays.toString(test));
        System.out.println(" Optimal O(1) Space: " + Arrays.toString(productExceptSelfOptimal(test)));
        System.out.println(" Two Arrays O(N): " + Arrays.toString(productExceptSelfTwoArrays(test)));
        System.out.println();
    }
}

```

Summary of Complexity Approach Time Complexity Auxiliary Space Why it works / Trade-offs Brute Force

(Nested Loops) $O(N^2)$ $O(1)$ Multiply every other number for every index. Guaranteed "Time Limit Exceeded".

Two Arrays (Left/Right) $O(N)$ $O(N)$ Highly intuitive. Pre-calculates all left sides and right sides, then merges them.

Optimal (On-the-fly) $O(N)$ $O(1)$ Most optimal. Achieves the exact same math as the Two Array approach, but manages state using simple integer variables and the output array itself.

Interview Tip: When explaining this to an interviewer, avoid jumping straight into the $O(1)$ space solution. Start by saying: "If I couldn't use division, my first thought would be to map out the product of everything to the left of an index in one array, and the product of everything to the right in another array. Then I could just multiply those arrays together index-by-index in $O(N)$ time. However, I can actually optimize that memory down to $O(1)$ by doing it in a two-pass calculation directly on the output array..." This shows a beautiful, logical progression of thought.

The "First Non-Repeating Element" problem is a classic interview question. It is most commonly asked in two formats: finding the first non-repeating character in a String (LeetCode 387) or the first non-repeating integer in an Array. The core logic for both is identical: we need to count the frequency of each element and then find the first one that has a count of exactly 1. Here are the most optimal ways to solve both variations, structured to fit perfectly into your project.

1. Optimal Approach for Strings: Frequency Array
If you are dealing with a String (which usually consists of standard ASCII characters), using a simple integer array as a frequency map is vastly faster and more memory-efficient than using a Java HashMap. We do one pass to count the characters, and a second pass to find the first character with a count of 1.

2. Optimal Approach for Arrays: LinkedHashMap
If you are dealing with an array of integers (where numbers can be incredibly large or negative), an array-based frequency map won't work. Instead, we use a LinkedHashMap. A standard HashMap does not maintain insertion order, meaning we would have to loop through the entire original array a second time to find the first non-repeating number. A LinkedHashMap maintains the exact order elements were inserted. This allows our second loop to only iterate over the unique elements, which is a massive performance boost if the original array was huge but only contained a few distinct numbers.

The CodeJava package
com.codes.java.codingquestions.arrays;

```
import java.util.Arrays;
import java.util.LinkedHashMap;
import java.util.Map;

public class FirstNonRepeating {

    // 1. Optimal Approach for Strings (ASCII Frequency Array)
    // Time: O(N), Space: O(1) - The array size is fixed at 256 regardless of input size
    public static char firstNonRepeatingChar(String s) {
        if (s == null || s.isEmpty()) {
            return '_'; // Return a default placeholder for empty/null strings
        }

        // Assuming standard ASCII characters.
        // (Use 26 if the interviewer specifies lowercase English letters only)
        int[] charCounts = new int[256];

        // Pass 1: Count frequencies
        for (int i = 0; i < s.length(); i++) {
            charCounts[s.charAt(i)]++;
        }

        // Pass 2: Find the first character with a frequency of 1
        for (int i = 0; i < s.length(); i++) {
            if (charCounts[s.charAt(i)] == 1) {
                return s.charAt(i);
            }
        }
    }
}
```



```

return '_'; // Return placeholder if all characters repeat
}

// 2. Optimal Approach for Integer Arrays (LinkedHashMap)
// Time: O(N), Space: O(U) where U is the number of unique elements
public static int firstNonRepeatingInt(int[] nums) {
    if (nums == null || nums.length == 0) {
        throw new IllegalArgumentException("Array cannot be empty");
    }

    // LinkedHashMap preserves the order of insertion
    Map<Integer, Integer> counts = new LinkedHashMap<>();

    // Pass 1: Count frequencies
    for (int num : nums) {
        counts.put(num, counts.getDefault(num, 0) + 1);
    }

    // Pass 2: Iterate through the map (which is in insertion order)
    for (Map.Entry<Integer, Integer> entry : counts.entrySet()) {
        if (entry.getValue() == 1) {
            return entry.getKey();
        }
    }

    throw new RuntimeException("No non-repeating element found");
}

// Test Cases
public static void run() {
    System.out.println("\n--- First Non-Repeating Element ---");

    // --- String Tests ---
    System.out.println("[String Tests]");
    String[] stringTests = {
        "leetcode",    // 'l' is first
        "loveleetcode", // 'v' is first
        "aabb",        // None -> '_'
        "z"            // 'z' is first
    };

    for (String test : stringTests) {
        System.out.println(" Input: \"" + test + "\" -> First Unique: " + firstNonRepeatingChar(test) + "");
    }

    System.out.println("\n[Integer Array Tests]");
    // --- Integer Array Tests ---
    int[][] arrayTests = {

```

```

    {9, 4, 9, 6, 7, 4}, // 9 and 4 repeat. 6 is first unique.
    {2, 3, 4, 2, 3, 4}, // All repeat -> Exception/Error
    {10, 10, 10, 5}, // 5 is first unique
    {-1, 2, -1, 3} // -1 repeats, 2 is first unique
};

for (int[] test : arrayTests) {
    System.out.print(" Input: " + Arrays.toString(test) + " -> First Unique: ");
    try {
        System.out.println(firstNonRepeatingInt(test));
    } catch (RuntimeException e) {
        System.out.println("None");
    }
}
}

```

Summary of Complexity Approach Use Case Time Complexity Auxiliary Space Why it works / Trade-offs

Nested Loops Either $O(N^2)$ or $O(1)$ Checks every element against every other element. Usually results in a rejection in an interview.

Standard HashMap Arrays $O(N)$ $O(N)$ Requires you to loop through the entire original array a second time to ensure you find the "first" element in order.

LinkedHashMap Arrays $O(N)$ $O(N)$ Optimal for general data. Maintains order automatically. Second loop only iterates over unique elements, saving time.

Frequency Array Strings $O(N)$ $O(1)$ Optimal for Strings. Arrays are incredibly fast to access, and since the alphabet size is constant (e.g., 256), space complexity is strictly $O(1)$.

Interview Tip: When solving the Array version, explicitly mention your choice of LinkedHashMap over a standard HashMap. Say: "I could use a regular HashMap, but I would have to loop through the original array again to guarantee I find the first element in chronological order. By using a LinkedHashMap, I can just iterate through the map's keys directly, which guarantees chronological order and is much faster if there are many duplicate numbers."